

Carolus / RoboMaster S1 — Complete Technical Manual

August 3, 2026

Abstract

This manual is self-sufficient: following it from Chapter 2 onward requires no other document, inherited tutorial, or vendor PDF. It covers the complete path from unboxing a rooted RoboMaster S1 and a Raspberry Pi 4B to a running Carolus/uVGS-2 pose-estimation pipeline with autonomous beacon docking, using only the corrected, currently-working configuration of this project — not the historical or inherited versions of any script, launch file, or procedure. Every program listed here is the version that actually runs today.

Contents

1	Introduction	2
1.1	What this manual covers	2
1.2	Hardware	2
1.3	Software stack	2
1.4	What Carolus is	3
1.5	The four operating states	3
1.6	How to use this manual	3
2	Powering On the Robot	5
2.1	Before you start	5
2.2	Power-on sequence	5
2.3	Confirming the robot is powered on	6
2.4	Confirming the robot is rooted	6
2.5	Which camera is mounted	6
3	Rooting the S1	7
3.1	What the hack does	7
3.2	Step-by-step procedure	7
3.2.1	What you need	7
3.2.2	Procedure	8
3.3	A pitfall to know before rooting	9
4	Setting Up the Raspberry Pi	10
4.1	Flashing Ubuntu Server	10
4.2	Desktop environment (optional)	10
4.3	Enabling headless SSH access	11
4.4	Building Python 3.7 from source	11
4.5	Virtual environment and the RoboMaster SDK	12
4.6	Installing ROS Noetic	12
4.7	Getting this project's code onto the Pi	13
4.8	A pitfall to expect at first launch	13
5	Network Setup (RNDIS)	14
5.1	Bringing up the link	14
5.2	Verifying the link	14

5.3	A quick automated alternative	14
5.4	Lab PC hostname resolution	15
5.5	Summary of addresses used in this manual	15
6	The Camera and Robot Bridge	16
6.1	Design: a single SDK connection	16
6.2	Starting the bridge	16
6.3	What it connects to	16
6.4	Verifying it is running	17
6.5	Gimbal yaw: two signals, two different meanings	17
6.6	The internal camera vs. an external D455	18
7	Camera Calibration	19
7.1	Current values in use	19
7.2	Recommended method: a checkerboard with the MATLAB Camera Calibration Toolbox	19
7.3	Alternative / complementary method: Kalibr	20
7.3.1	Target	20
7.3.2	Installing Kalibr	20
7.3.3	Capturing data and running the calibration	21
7.3.4	Reading the result	21
7.4	Camera-IMU time offset	21
8	Building and Launching Carolus	22
8.1	Where Carolus runs, and why it should move to the Pi	22
8.1.1	The cost of running it on the PC	22
8.1.2	The intended target: build and run on the Raspberry Pi	23
8.2	Getting this project's code onto the lab PC	23
8.2.1	Requesting access	23
8.2.2	Setting up an SSH key	23
8.3	Building on Ubuntu 20.04	24
8.4	Building on Ubuntu 22.04 (unsupported, requires fixes)	24
8.4.1	Fix 1: glibc symlinks	24
8.4.2	Fix 2: libpython3.8 symlink	25
8.4.3	Fix 3: Boost 1.71 (real libraries, not symlinks)	25
8.4.4	Fix 4: ICU 67	25
8.4.5	Building, after the four fixes	26
8.4.6	Fix 5: LD_PRELOAD at launch (a runtime fix, not a build fix)	26
8.4.7	If an additional ROS package needs to be built from source	26
8.5	Launching Carolus	26
8.5.1	The hardware profile	27
8.5.2	Launching	27
8.6	Verifying it is running	28
8.7	Beacon geometry and blob-filtering parameters	28

Chapter 1

Introduction

1.1 What this manual covers

This is the complete, self-contained setup and operation manual for the Carolus/RoboMaster S1 platform: a DJI RoboMaster S1 ground robot, rooted to expose the RoboMaster EP SDK, carrying a Raspberry Pi 4B at the turret position, running Carolus (also called uVGS-2) — a 4-LED beacon pose-estimation system — to autonomously search for, align to, approach, and dock at a fixed beacon.

Everything in this manual has been executed and verified on this exact hardware. Where a step has not been physically re-verified on this specific unit but is inherited from a documented, working precedent (e.g. a rooting procedure that predates this project), that is stated explicitly at the point it occurs.

1.2 Hardware

- **Robot:** DJI RoboMaster S1, rooted via the `s1_sdk_hack_v0.0.5` community hack, which exposes the RoboMaster EP SDK (chassis drive, gimbal control, onboard camera, wheel odometry, IMU) over a local API not officially released for the S1.
- **Companion computer:** Raspberry Pi 4B, mounted at the turret, connected to the robot over USB via RNDIS (a virtual Ethernet link over USB). The Pi holds the single connection to the robot's SDK and runs the camera/beacon-tracking node.
- **Lab PC:** runs the operator GUI and, in the currently-validated configuration, the Carolus pose-estimation node. Not physically connected to the robot; communicates with the Pi over the local network. **Carolus is intended to move onto the Raspberry Pi** so the robot can operate without a network link to the PC — see Section 8.1.
- **Camera:** the S1's own internal camera (built into the gimbal/head), streamed over the SDK. This project does not use an external Intel RealSense D455 — some inherited reference material describes a D455-based pipeline (see Chapter 7's calibration discussion); that material is noted where it applies to calibration *technique*, not to this robot's actual camera hardware.

1.3 Software stack

- **Operating systems:** Raspberry Pi runs Ubuntu 20.04.6 LTS (aarch64). The lab PC runs Ubuntu 22.04.5 LTS (x86_64) — not the officially supported target for ROS Noetic, which requires local

workarounds documented in Chapter 8.

- **Middleware:** ROS Noetic (1.17.4) on both machines.
- **Robot control:** the official `robomaster` Python SDK, requiring Python 3.7 specifically.
- **Vision:** Carolus/uVGS-2, a C++ ROS node built on OpenCV, Eigen3, and Ceres Solver, performing 4-LED blob detection and a Perspective-4-Point (P4P) pose solve.

1.4 What Carolus is

Carolus (internally named uVGS-2, Unified Visual Guidance System version 2) is a monocular pose-estimation system originally developed for NASA’s Astrobee free-flying robot aboard the International Space Station, derived from the earlier SVGS system. It detects four illuminated beacon points (LEDs) in a camera image, identifies which detected blob corresponds to which known 3D point on the physical beacon, and solves a Perspective-4-Point optimization (via Ceres Solver, not OpenCV’s `solvePnP` family) to recover the beacon’s full 6-DOF pose relative to the camera.

Carolus’s own native coordinate frame is left-handed. This is not obvious from the code and is stated here because it matters for every transform downstream: converting a Carolus pose or orientation into ROS’s right-handed convention (REP-103) requires a true change-of-basis on the rotation, not a component-wise permutation of a quaternion’s vector part — the latter is only valid for a proper (determinant +1) rotation, and a left-to-right-handed conversion is improper (determinant -1). The correct method (quaternion composition, not permutation) is given at the point it is needed in this manual.

1.5 The four operating states

Once running, the robot’s autonomous behavior is a state machine:

SEARCH The gimbal sweeps without moving the chassis, looking for the beacon. If not found from the current heading, the chassis rotates to a new heading and the gimbal sweeps again (a “fan search” pattern).

ALIGN Once the beacon is detected, the chassis rotates in place until it faces the beacon.

APPROACH The gimbal tracks the beacon while the chassis advances along the same heading.

STOP Triggered once within a target stopping distance of the beacon.

A separate, more deliberate **docking** behavior performs a calibrated, verified approach-and-stop sequence rather than the lighter continuous SEARCH/ALIGN/APPROACH loop. It runs through this project’s own operator tooling, not yet covered in this manual (see the note at the end of Chapter 8).

1.6 How to use this manual

Follow the chapters in order for a from-scratch setup: power the robot on and confirm it is rooted (Chapter 2), root it if it is not (Chapter 3), set up the Raspberry Pi (Chapter 4), bring up the network link (Chapter 5), bring up the camera pipeline (Chapter 6), calibrate the camera (Chapter 7), then build and launch Carolus (Chapter 8). At that point the core pipeline is running and can be driven

manually over the topics named in Chapter 6; the operator GUI and autonomous docking behavior are this project's own tooling on top of it, documented separately.

Every command, parameter value, and code listing in this manual is the *current, corrected* version — the one that actually runs on this project today.

Chapter 2

Powering On the Robot

Runs on: the **robot itself** (physical handling). The Raspberry Pi and lab PC are not involved until Chapter 4.

2.1 Before you start

- The S1’s smart battery, charged. Charging is required before first use even if the battery shows some charge — this “wakes up” the battery.
- The S1 charger, plugged into mains (100–240 V, 50/60 Hz). A full charge takes approximately 1 h30.
- The battery only charges between 5 °C and 45 °C.
- Do not touch the motors, motor mounting plate, or the inside of the Mecanum wheels immediately after powering the robot off — they can be hot.
- Do not attempt to charge a battery that has dropped below 1 V per cell; this poses a fire risk.
- Use the official DJI charger only.

2.2 Power-on sequence

The battery compartment is behind the chassis’s rear armor panel, so the sequence is: open, insert, power on, close.

1. Press the rear armor release button to open the chassis’s rear cover.
2. Insert the smart battery into its compartment, making sure it is firmly seated — a poorly seated battery gives poor contact and can cause the robot to lose battery information.
3. Press and hold the power button (on the battery itself) for more than two seconds to power on.
4. Close the chassis’s rear armor.

The Raspberry Pi at the turret is a separate power domain from the robot chassis; there is no required power-on order between the two, and the USB/RNDIS cable connecting them is for the SDK link (Chapter 5), not for powering either device.

2.3 Confirming the robot is powered on

Expected light signals at power-on:

- **Gimbal (head) LED:** blinks cyan, counterclockwise.
- **Chassis LED:** solid cyan.
- **Motion controller LED:** blinks blue slowly — this means the motion controller is working normally.

2.4 Confirming the robot is rooted

This project assumes a rooted S1 exposing the RoboMaster EP SDK. Without the root hack (Chapter 3), SDK commands (chassis, gimbal, camera) will not respond, even though the robot otherwise powers on and behaves normally through the stock RoboMaster app.

Audible cue at boot: a correctly rooted S1 emits two chimes instead of the usual single chime. The hack persists across power cycles — it does not need to be reapplied every time the robot is powered on.

Concrete test, once the Pi is connected over RNDIS (Chapter 5): the camera/beacon node (`rm_cam_beacon.py`, Chapter 6) opens the one and only SDK connection this project uses:

```
1 self.ep = robot.Robot()  
2 self.ep.initialize(conn_type="rndis")
```

- **Success:** the node's log shows the connection succeeding, and the robot responds to `Twist` commands published on `/carolus/cmd_vel` (Chapter 6).
- **Failure:** the SDK connection fails or times out. First confirm the network link itself (ping 192.168.42.2 from the Pi, Chapter 5). If the network link is fine but the SDK connection still fails, redo the power-on and listen carefully for the double chime — a single chime means the root hack is not active.

Do not use any script other than `rm_cam_beacon.py` to test this. Two independent scripts each attempting `ep.initialize(conn_type="rndis")` at the same time silently blocks robot motion after a few commands, with no error surfaced — this failure mode is why this project uses exactly one script holding the SDK connection at any time (see Chapter 6).

2.5 Which camera is mounted

This project uses the S1's own internal camera, not an external Intel RealSense D455. Physically: a D455 is an external rectangular camera housing mounted in addition to the robot's own head; if there is no such external camera, the internal camera is in use. In software, once `rm_cam_beacon.py` is running (Chapter 6), the internal camera feed appears on `/camera/color/image_raw` immediately, with no separate D455 driver (`rs_camera.launch`) needed.

Chapter 3

Rooting the S1

Runs on: a **Windows or macOS computer** connected to the robot by micro-USB. Not the Raspberry Pi, not the Ubuntu lab PC — the RoboMaster app required for this step does not run on Linux.

If Chapter 2’s double-chime check already confirmed the robot is rooted, this chapter is informational only — do not re-run it.

3.1 What the hack does

DJI’s RoboMaster app exposes a limited, sandboxed Python scripting environment (“the Lab”). The S1’s Python subsystem runs with root privileges internally, but DJI restricts which modules a Lab script can import via a whitelist attached to the `__builtins__` object. Several of the whitelisted modules (needed for legitimate scripting features) still hold a reference to the real, unrestricted `__builtins__` internally. That reference can be recovered and used to import `subprocess`, which can then invoke a shell script already present on the robot’s filesystem (`adb_en.sh`) to enable `adb` and open a root shell over USB.

Once a root shell is available, a patch is pushed to the robot’s filesystem that installs the RoboMaster EP SDK’s full command set (the EP is DJI’s higher-end sibling to the S1; underneath, the two share nearly identical firmware) onto the S1, which DJI has never officially released to S1 owners.

Warning, carried over from the hack’s own documentation, to be taken literally: this has the potential to break the robot. It did not break this project’s unit, but there is no guarantee for any given unit.

3.2 Step-by-step procedure

3.2.1 What you need

- A computer running Windows or macOS (the RoboMaster app requires one of these to configure the S1; the Raspberry Pi is not involved in this chapter).
- Android SDK Platform-Tools (`adb.exe` and friends) — unzip anywhere, remember the path.
- A micro-USB cable.
- The RoboMaster app, with a DJI account.

3.2.2 Procedure

1. Connect the computer to the S1's Intelligent Controller Micro-USB port with the micro-USB cable. Ensure no other Android device is connected over USB during this procedure.
2. Power on the robot (Chapter 2) and connect the RoboMaster app to it via its own Wi-Fi hotspot (visible on top of the gimbal, with a password shown alongside it).
3. In the app: **Lab** → **DIY Programming** → select **Python** on the top bar → **New** to create a new program.
4. Paste the following into the program editor exactly. This is the current, working root script for this hack version — do not use an older variant that imports `'subprocess'` as a literal string or reads from `rm_log` instead of `rm_define`; a firmware update closed that specific path.

```
1 def root_me(module):
2     __import__=rm_define.__dict__['__builtins__']['__import__']
3     return __import__(module, globals(), locals(), [], 0)
4
5 builtins=root_me('builtins')
6 x=root_me('sub'+ 'process')
7 proc=x.Popen('/system/bin/adb_en.sh', shell=True, executable='/system/bin/
8     sh',
9             stdout=x.PIPE, stderr=x.PIPE)
```

5. Run the program from within the Lab. A correct run shows no compilation errors and the console prints **Execution Complete**. Do not close the RoboMaster app.
6. On the computer, open a terminal in the folder containing the Android Platform-Tools and run:

```
adb devices
```

The S1 should appear in the device list.

7. Confirm root access:

```
adb shell
```

8. With the root shell open, from a second terminal, in the folder containing the hack's distribution files (`dji_scratch`, `dji.json`, `dji_hdvt_uav`, `dji_sys`, `patch.sh`), push the patch:

```
adb shell rm -rf /data/dji_scratch
adb push dji_scratch /data/.
adb push dji.json /data/.
adb push dji_hdvt_uav /data/.
adb push dji_sys /data/.
adb shell chmod 755 /data/dji_hdvt_uav
adb shell chmod 755 /data/dji_sys
adb push patch.sh /data/.
```

9. Restart the S1 fully (hold the power button until it turns off, then hold it again to turn back on). On boot, listen for **two chimes instead of one** — this confirms the hack is active (Chapter 2's check).
10. Close the RoboMaster app and power off the robot; the setup for this chapter is complete.

3.3 A pitfall to know before rooting

Starting with hack version 0.0.3, `adb` is re-enabled by default in the patch, which **disables RNDIS support** — the exact link this project’s Pi↔robot connection depends on (Chapter 5). If RNDIS stops working after rooting, comment out the `adb` line in `patch.sh` before pushing the patch, or re-push a corrected `patch.sh` and restart the robot again.

Chapter 4

Setting Up the Raspberry Pi

Runs on: the **Raspberry Pi 4B** (mounted on the robot), with a keyboard/monitor for first boot or over SSH from the lab PC afterwards.

This chapter takes a blank Raspberry Pi 4B to a working Ubuntu 20.04 installation with Python 3.7, the RoboMaster SDK, ROS Noetic, and SSH access. If the Pi is already set up, skip to Chapter 5.

4.1 Flashing Ubuntu Server

1. Download the Ubuntu Server ARM64 image from [`https://cdimage.ubuntu.com/releases/focal/release/`](https://cdimage.ubuntu.com/releases/focal/release/) (`ubuntu-20.04.5-preinstalled-server-arm64+raspi.img.xz`).

2. Decompress it:

```
unxz ubuntu-20.04.5-preinstalled-server-arm64+raspi.img.xz
```

3. Identify the SD card device (do this carefully — writing to the wrong device destroys data on it):

```
lsblk
```

4. Flash it:

```
sudo dd if=ubuntu-20.04.5-preinstalled-server-arm64+raspi.img of=/dev/sdX  
bs=4M status=progress conv=fsync
```

5. Boot the Pi with the card inserted, plus keyboard, mouse, display, and an Ethernet cable for first boot. Default credentials: `ubuntu/ubuntu` (you will be asked to change the password on first login).

4.2 Desktop environment (optional)

Only needed if you want a graphical session on the Pi itself (not required for this project's normal operation, which runs headless):

```
sudo apt update  
sudo apt install ubuntu-desktop  
sudo reboot
```

4.3 Enabling headless SSH access

If SSH was not already enabled at flash time (Raspberry Pi Imager's advanced options support this directly), enable it on first boot:

```
sudo systemctl enable ssh
sudo systemctl start ssh
sudo systemctl status ssh    # optional: confirm it's running
```

Find the Pi's IP address on the local network:

```
hostname -I
```

Connect from the client machine, first with a password, then switch to key-based login:

```
# First connection (password):
ssh <username>@<pi_ip>

# On the client: generate a key
ssh-keygen -t ed25519 -C "client-key"

# Copy it to the Pi
ssh-copy-id <username>@<pi_ip>

# Test key-based login (no password prompt expected)
ssh <username>@<pi_ip>
```

A convenient alias, in ~/.ssh/config on the client:

```
1 Host raspberry
2   HostName <pi_ip>
3   User <username>
4   IdentityFile ~/.ssh/id_ed25519
```

Note the exact directive name: `IdentityFile`, not `IdentifyFile` — a common typo that OpenSSH silently rejects the config over.

Connect with `ssh raspberry` from then on.

Optional hardening, once key-based login is confirmed working:

```
# Firewall
sudo apt install -y ufw
sudo ufw allow OpenSSH
sudo ufw enable

# Disable password authentication (only after keys work!)
sudo nano /etc/ssh/sshd_config
# Ensure: PasswordAuthentication no
#         PubkeyAuthentication yes
sudo systemctl restart ssh
```

4.4 Building Python 3.7 from source

The RoboMaster SDK requires Python 3.7 specifically; Ubuntu 20.04 ships a newer default Python, so 3.7 is built from source into a separate location rather than replacing the system Python.

```

sudo apt update
sudo apt install -y build-essential libssl-dev zlib1g-dev \
    libreadline-dev libsqlite3-dev libbz2-dev libexpat1-dev \
    liblzma-dev tk-dev libffi-dev uuid-dev

cd ~/Python-3.7.17/
./configure --enable-optimizations
make -j4
sudo make altinstall

```

Using `altinstall` (not `install`) is deliberate: it installs as `python3.7` alongside the system Python, without overwriting it.

Confirm the SSL module built correctly (a common source of a broken venv later if skipped):

```
python3.7 -c "import ssl; print(ssl.OPENSSL_VERSION)"
```

4.5 Virtual environment and the RoboMaster SDK

```

python3.7 -m venv env
source env/bin/activate
pip install --upgrade pip
pip install robomaster numpy
pip install "opencv-python==4.5.5.64"

```

The pinned OpenCV version is a historically validated compatible version for this SDK/Python combination on this hardware; it is not a hard requirement enforced elsewhere in this manual, and later versions have since been used successfully on the lab PC (Chapter 8) without issue.

4.6 Installing ROS Noetic

```

sudo sh -c 'echo "deb http://packages.ros.org/ros/ubuntu \
$(lsb_release -sc) main" > /etc/apt/sources.list.d/ros-latest.list'
sudo apt install curl
curl -s https://raw.githubusercontent.com/ros/rosdistro/master/ros.asc \
    | sudo apt-key add -
sudo apt update

# Full desktop version:
sudo apt install ros-noetic-desktop-full
# --- or, if the Pi struggles with the desktop version's resource use ---
sudo apt install ros-noetic-ros-base

echo "source /opt/ros/noetic/setup.bash" >> ~/.bashrc
source ~/.bashrc

sudo apt install python3-rosdep python3-rosinstall \
    python3-rosinstall-generator python3-wstool build-essential
sudo rosdep init
rosdep update

```

On this project's Pi, the lighter `ros-noetic-ros-base` is what actually runs day to day — the Pi only needs to run `rm_cam_beacon.py` (Chapter 6), not a full desktop ROS toolchain.

4.7 Getting this project's code onto the Pi

The Pi needs this project's own workspace — specifically `carolus_ws/src/robomaster_cam/`, which holds `rm_cam_beacon.py` (Chapter 6) and its companion scripts. Clone the project's repository directly onto the Pi:

```
git clone git@github.com:Arracheur2Bonnet/robotmaster.git carolus_ws
```

This is a private repository; you need an SSH key registered against it (ask the project supervisor for collaborator access if you do not already have it) — do not substitute a personal access token pasted into the clone URL for this. If cloning over SSH is not set up, an HTTPS clone with your own GitHub credentials works the same way:

```
git clone https://github.com/Arracheur2Bonnet/robotmaster.git carolus_ws
```

No build step is needed on the Pi side for `robomaster_cam` — it is a pure-Python package, run directly (Chapter 6).

4.8 A pitfall to expect at first launch

The RNDIS network interface (Chapter 5) does not come up automatically at Pi boot, even once everything above is configured correctly. This is expected — Chapter 5 gives the exact two commands to run at the start of every session.

Chapter 5

Network Setup (RNDIS)

Runs on: the **Raspberry Pi** for the interface commands; one step at the end runs on the **lab PC** and is marked as such.

The robot exposes an RNDIS virtual Ethernet interface over the same USB cable used for the SDK connection. The robot holds a fixed address, 192.168.42.2; the Raspberry Pi must be configured on the same subnet.

5.1 Bringing up the link

Run these two commands at the start of every session, on the Pi. The interface does not persist across Pi reboots — this is expected behavior on this hardware, not a fault, and there is no step elsewhere in this manual that avoids needing it.

```
sudo ip link set eth1 up
sudo ip addr add 192.168.42.3/24 dev eth1
```

If the interface is not named `eth1` on your system, identify it first:

```
ip -br addr
# look for eth1 / usb0 / enx<MAC> -- whichever one is meant to carry
192.168.42.x
```

Never assign 192.168.42.2 to the Pi — that address belongs to the robot.

5.2 Verifying the link

```
ping -c 4 192.168.42.2
```

Expect 0% packet loss, sub-millisecond round-trip time once the interface is correctly up.

5.3 A quick automated alternative

Rather than typing the two commands above by hand every session, they can be scripted:

```
sudo ip link set eth1 up && sudo dhclient eth1
```

This project's own operator tooling (not covered in this manual) runs the equivalent of the first two-command form automatically as part of its first launch step, over SSH from the lab PC.

5.4 Lab PC hostname resolution

If any ROS tool on the lab PC (e.g. `rqt_image_view`) shows a black image or errors with something like `Couldn't find AF_INET address for [ubuntu]`, add the Pi's hostname to `/etc/hosts` on the lab PC:

```
echo "192.168.0.103_ubuntu" | sudo tee -a /etc/hosts
```

Unlike the RNDIS interface on the Pi, this fix is permanent on the lab PC and does not need to be repeated.

5.5 Summary of addresses used in this manual

Device	Address
Robot (over RNDIS)	192.168.42.2
Raspberry Pi (over RNDIS, to the robot)	192.168.42.3
Raspberry Pi (on the local network, to the lab PC)	192.168.0.103

The last of these is a DHCP-assigned local-network address and may differ on a different network; find it with `hostname -I` run on the Pi (Chapter 4).

Chapter 6

The Camera and Robot Bridge

Runs on: the **Raspberry Pi** — this is the only machine that ever connects to the robot SDK. The verification commands at the end run from the **lab PC**.

6.1 Design: a single SDK connection

The Pi runs exactly one script that ever connects to the robot's SDK: `rm_cam_beacon.py` (in `carolus_ws/src/robomaster_cam`). It is not just a camera capture script — it is this project's single bridge to the entire SDK: camera stream, chassis drive, gimbal control, odometry, and IMU all go through this one script and its one connection.

This is a deliberate design choice, not an incidental one. Two independent scripts each holding their own `ep.initialize(conn_type="rndis")` connection at the same time silently blocks robot motion commands after a few calls, with no error surfaced. **Never run any other script that calls `robot.Robot().initialize(...)` while `rm_cam_beacon.py` is running.**

6.2 Starting the bridge

On the Pi, with the RNDIS link up (Chapter 5) and the Python 3.7 virtual environment active (Chapter 4):

```
source ~/env/bin/activate
python3.7 carolus_ws/src/robomaster_cam/scripts/rm_cam_beacon.py
```

In normal operation this is launched automatically by this project's own operator tooling, over SSH from the lab PC — it is not usually started by hand.

6.3 What it connects to

```
1 self.ep = robot.Robot()
2 self.ep.initialize(conn_type="rndis")
3 self.chassis = self.ep.chassis
4 self.gimbal = self.ep.gimbal
5 self.cam = self.ep.camera
6 self.cam.start_video_stream(display=False)
```

From there it subscribes to the SDK’s own push-data channels (`sub_position`, `sub_attitude`, `sub_velocity`, `sub_imu`, `sub_angle` for the gimbal), converts each callback into a ROS message, and publishes:

Topic	Content
<code>/camera/color/image_raw</code>	Camera frames (<code>sensor_msgs/Image</code>)
<code>/odom</code>	Wheel odometry, republished from the SDK
<code>/imu</code>	Raw IMU (accelerometer + gyroscope), 50 Hz
<code>/carolus/gimbal_yaw_rel</code>	Gimbal yaw relative to the chassis
<code>/carolus/gimbal_yaw_ground</code>	Gimbal yaw in the world frame (absolute)

and subscribes to a small set of control topics used by this project’s own higher-level tooling (not covered in this manual): `/pose` (Carolus’s own output, consumed here to drive autonomous behavior), `/carolus/mode`, `/carolus/cmd_vel`, `/carolus/gimbal_vel`, `/carolus/wheels`, `/carolus/gimbal_lock`, `/carolus/gimbal_lock_period`, and `/carolus/gimbal_recenter`.

6.4 Verifying it is running

From the lab PC, with the ROS master reachable:

```
rostopic list | grep camera
rostopic hz /camera/color/image_raw
rostopic echo -n1 /odom
rostopic echo -n1 /imu
```

Expect a steady rate on `/camera/color/image_raw` (roughly 10–15 Hz) and at least one non-empty message on `/odom` and `/imu`. If the camera topic is silent, confirm the RNDIS link (Chapter 5) before suspecting the script itself — a dead network link and a dead camera stream look identical from this check alone.

6.5 Gimbal yaw: two signals, two different meanings

`/carolus/gimbal_yaw_rel` and `/carolus/gimbal_yaw_ground` are not interchangeable, and confusing them is a documented source of bugs in this project’s history:

- **`gimbal_yaw_ground`** is the gimbal’s heading in the world frame. The S1’s gimbal is yaw-stabilized: when the chassis rotates without an explicit gimbal command, the gimbal holds its absolute heading. This value stays essentially constant across a chassis rotation.
- **`gimbal_yaw_rel`** is the gimbal’s angle *relative to the chassis*. Because the gimbal holds its world heading while the chassis turns underneath it, this value *absorbs* the chassis’s rotation — it is not a stable “how far off is the beacon” signal across a chassis maneuver, it is a signal that actively tracks how much the chassis has turned since the gimbal last held still relative to it.

Use `gimbal_yaw_ground` when you need to know the camera’s absolute heading in the world. Use `gimbal_yaw_rel` only for what it actually is: the angle between the gimbal and the chassis at this instant, not a proxy for beacon bearing across a chassis rotation.

6.6 The internal camera vs. an external D455

This robot uses its own internal camera exclusively. If, when following any inherited reference material, you encounter instructions to launch `rs_camera.launch` or install `librealsense`, that material concerns a different camera than the one on this robot and does not apply here — see the calibration discussion in Chapter 7 for where that material remains genuinely useful (the calibration *technique*, not the camera driver).

Chapter 7

Camera Calibration

Runs on: the lab PC (or any machine with MATLAB / a Kalibr install). Images and rosbags come from the robot camera over the network.

7.1 Current values in use

The intrinsics and distortion coefficients currently loaded by `testcarolus.launch` (Chapter 8) are:

Parameter	Value
f_x	546.1957
f_y	547.0838
c_x	575.6041
c_y	372.1876
Distortion (plumb_bob: k_1, k_2, p_1, p_2, k_3)	-0.1479, 0.1452, -0.0023, 0.0025, 0.0070

These come from `robomaster_cam/config/camera_info.yaml` (`camera_name: robomaster_s1`, 1280×720), validated pragmatically — Carolus produces a stable, coherent pose with these values in practice. They are **not** the result of a formal metric calibration campaign (checkerboard/AprilGrid target, dedicated calibration tool run to convergence with a reported reprojection error). If you are setting up a new unit, or want a rigorous calibration, follow the procedure below rather than reusing these values — they are specific to this project’s camera and are not guaranteed to transfer to a different physical unit.

7.2 Recommended method: a checkerboard with the MATLAB Camera Calibration Toolbox

This procedure has not yet been run against this project’s own camera — the values in the previous section are what is actually loaded today. It is included here as the recommended path for whoever runs the next calibration, based on a direct comparison against Kalibr on a comparable camera (Section 7.3).

1. Print a checkerboard target with known square size (25 mm squares is a proven working size for this class of camera).
2. Capture a set of images of the board with the robot’s camera (Chapter 6): at least 10 images, 15 or more for better accuracy. The board should occupy roughly 20–25% of the frame area, and be

tilted more than 45° from the camera’s optical axis in each shot to avoid detection errors. Capture from a wide variety of positions and orientations across the field of view.

3. In MATLAB (desktop or MATLAB Online, no additional software required): open the **Camera Calibration app** (Apps tab), add the captured images, select the checkerboard pattern with the correct square size, and enable **tangential distortion** and **three radial distortion coefficients** (this camera class shows moderate wide-angle distortion that the default two-coefficient model under-fits).
4. Run the calibration. Export the resulting coefficients to the MATLAB workspace and generate the calibration script — this script contains the intrinsic matrix, distortion coefficients, and reprojection errors in a form that can be transcribed directly into `testcarolus.launch`’s `fx/fy/cx/cy/distortion` parameters (Chapter 8).

7.3 Alternative / complementary method: Kalibr

Kalibr (ETH Zürich) is the standard tool if a camera–IMU joint calibration is needed (required for any future MINS/visual-inertial integration, not needed for Carolus’s own pose estimation, which does not use the IMU).

7.3.1 Target

A 6×6 AprilGrid, tag size 0.089 m, tag spacing $0.3 \times$ tag size (an $\sim 0.8 \times 0.8$ m printed board). Measure the actual printed square size and use the measured value, not the nominal one — even a $\sim 1\%$ sizing error biases the recovered scale.

```
1 target_type: 'aprilgrid'
2 tagCols: 6
3 tagRows: 6
4 tagSize: 0.089          # metres -- MEASURE your own printed board
5 tagSpacing: 0.3         # ratio of gap to tag size
6 codeOffset: 0
```

7.3.2 Installing Kalibr

```
cd ~/camera_ws/src
git clone https://github.com/ethz-asl/kalibr.git
cd ~/camera_ws
rosdep install --from-paths src --ignore-src -y
sudo apt install python3-rosdep python3-rosinstall python3-rosinstall-
generator \
    build-essential python3-catkin-tools
sudo apt-get install libv4l-dev doxygen python3-igraph
sudo apt install python3-pip
pip install wxPython      # can take a long time -- be patient
catkin_make -DCMAKE_BUILD_TYPE=Release
source ~/camera_ws/devel/setup.bash
```

Do not separately clone `numpy_eigen` or `Schweizer-Messer` — Kalibr bundles its own copies, and cloning them independently into `src/` produces duplicate-package build errors. If you hit `ModuleNotFoundError: No module named 'sm'` after building, verify with:

```
python3 -c "import sm; print(sm.__file__)"
```

and if it fails, remove any independently-cloned copy of Schweizer-Messer, then rebuild:

```
rm -rf build/ devel/ install/  
catkin_make -DCMAKE_BUILD_TYPE=Release
```

7.3.3 Capturing data and running the calibration

Record a rosbag while moving the camera (or the robot itself, for a joint camera–IMU calibration) through the AprilGrid’s field of view for 60–90 seconds, exciting all six degrees of freedom (translate near/far/side to side, rotate about all three axes, not just yaw).

```
rosvarecord -O calib_data.bag /camera/color/image_raw
```

Run the calibration:

```
export KALIBR_MANUAL_FOCAL_LENGTH_INIT=1  
kalibr_calibrate_cameras \  
  --bag calib_data.bag --target target.yaml \  
  --models pinhole-radtan \  
  --topics /camera/color/image_raw \  
  --approx-sync 0.01
```

If Kalibr reports **Initialization of focal length failed**, provide a manual seed value (600 is a reasonable starting guess for a 640×480 -class image; scale proportionally for other resolutions).

7.3.4 Reading the result

Kalibr writes three files: a `.txt` summary, a `.pdf` of reprojection-error plots, and a `.yaml` with the calibration parameters. **Target: reprojection error under 0.2–0.5 pixels.** In practice, a completed run can land a little outside that target (observed: 0.59–0.67 px std on a comparable camera) without necessarily being unusable — treat the target as a goal to aim for, not a hard pass/fail gate, and judge the calibration by whether downstream pose estimates are stable and physically plausible.

7.4 Camera–IMU time offset

If a Kalibr camera–IMU joint calibration is run, its output `timeshift_cam_imu` is in **seconds**, with the convention $t_{\text{imu}} = t_{\text{cam}} + \text{shift}$. This matches OpenVINS’s own internal convention exactly — the value can be used directly with no sign flip if it is later consumed by an OpenVINS-based filter.

Chapter 8

Building and Launching Carolus

Runs on: the **lab PC** in the configuration documented here — see Section 8.1 for why, and for the planned move onto the Raspberry Pi.

Carolus (the ROS package name is `carolus_astrobee`) is built and run on the **lab PC** in the configuration this manual documents. It has no SDK connection of its own — it only consumes `/camera/color/image_raw` (published by `rm_cam_beacon.py` over the network from the Pi, Chapter 6) and publishes a pose.

8.1 Where Carolus runs, and why it should move to the Pi

Running Carolus on the lab PC is the currently-validated configuration, not the recommended end state. It has a structural cost that matters for autonomous operation, and the intended target is the Raspberry Pi. Both are stated here rather than presenting the current setup as settled.

8.1.1 The cost of running it on the PC

Two consequences follow directly from the split:

- **The robot cannot operate away from the PC.** Every pose estimate depends on a live network link to the lab PC, so the robot must stay within network reach at all times — incompatible with genuinely autonomous operation.
- **It is very likely the throughput bottleneck.** Carolus currently publishes poses at roughly 2.5 Hz on this setup. `/camera/color/image_raw` is published as *uncompressed sensor_msgs/Image* at 1280×720 — about 2.76 MB per frame in BGR8 — and every frame must cross the Pi→PC link before Carolus can process it. Published uVGS-2 benchmarks on comparable CPU hardware report throughput roughly 40× higher than what is observed here, which places the P4P/Ceres computation itself well within budget and points at the image transport path instead.

This is a reasoned diagnosis, not a measurement. The network link has not been instrumented directly; the conclusion rests on the frame size, the observed rate, and the published benchmark figures. It should be treated as the leading hypothesis, not an established fact.

8.1.2 The intended target: build and run on the Raspberry Pi

Running Carolus on the Pi removes the network hop entirely — the camera frames never leave the machine that captures them — which addresses both problems above at once. The target update rate for useful autonomous operation is **at least 10 pose updates per second**.

Two points support the move beyond the bottleneck argument:

- The Pi runs Ubuntu 20.04, which is the officially supported target for ROS Noetic. The lab PC runs 22.04 and needs the five compatibility fixes documented below purely as a consequence; a Pi-side build should not need any of them.
- Planned visual-inertial work (MINS) is likewise tested only on Ubuntu 18/20 with Melodic/Noetic, so the Pi qualifies and the 22.04 lab PC does not.

Not yet done, and deliberately not documented as if it were: Carolus has not yet been built or benchmarked on the Raspberry Pi on this project. Until a Pi-side build is compiled and its actual pose rate measured, the procedure below — which is what has genuinely been run — targets the lab PC. The build steps themselves are not architecture-specific (the same `catkin_make` sequence applies), so the Ubuntu 20.04 instructions in the next section are the relevant ones for a Pi build; the 22.04 fixes after them are lab-PC-specific and should be skipped.

8.2 Getting this project’s code onto the lab PC

Clone the project’s repository onto the lab PC. Cloning into a directory named `carolus_ws` gives a standard catkin workspace layout, with the ROS packages under `src/`: `src/carolus_node/` (the launch files and configuration), `src/libuvgs_astrobbee/` (Carolus’s own C++ source, already patched for this robot), `src/ff_msgs/` (a message-definition dependency), `src/robomaster_cam/` (the Pi-side bridge, Chapter 6), plus `cmake_shims/` (Ubuntu 22.04 build fixes, see below) and `shortcuts/` (this project’s operator tooling, not covered in this manual) at the top level:

```
git clone git@github.com:Arracheur2Bonnet/robotmaster.git carolus_ws
# or, over HTTPS with your own GitHub credentials:
# git clone https://github.com/Arracheur2Bonnet/robotmaster.git carolus_ws
```

Repository web address: <https://github.com/Arracheur2Bonnet/robotmaster>

8.2.1 Requesting access

This is a private repository. If you open the link above without having been granted access, GitHub returns **404 Not Found**, not “403 Forbidden” — this is deliberate on GitHub’s part (it avoids confirming that a private repository exists at all), so a 404 here means “you do not have access yet”, not “the link is wrong”.

To get access, ask the repository owner or the project supervisor to add your GitHub account as a collaborator (*Settings* → *Collaborators* → *Add people* on the repository). You will need to give them your GitHub username. Once added, you will receive an email invitation that must be accepted before cloning works.

8.2.2 Setting up an SSH key

Cloning over SSH (the first command above) needs an SSH key registered against *your own* GitHub account. This is a one-time setup per machine, and it is what replaces the practice — found in some

older inherited documents — of pasting a shared personal access token directly into a clone command. A shared token grants whoever holds it the full access rights of the account that created it, cannot be attributed to an individual, and stays valid in every copy of the document that carries it.

1. Generate a key on the machine that will clone the repository (the Raspberry Pi and the lab PC each need their own):

```
ssh-keygen -t ed25519 -C "your_email@example.com"
# Press Enter to accept the default path (~/.ssh/id_ed25519).
# A passphrase is optional but recommended.
```

2. Display the **public** half of the key — this is the only part that ever leaves the machine. Never share the file without the `.pub` extension:

```
cat ~/.ssh/id_ed25519.pub
```

3. On GitHub: *profile photo* → *Settings* → *SSH and GPG keys* → *New SSH key*. Give it a title identifying the machine (e.g. “RoboMaster Pi”), paste the full line printed above, and save.
4. Verify the key works before attempting the clone:

```
ssh -T git@github.com
```

A successful setup answers with `Hi <username>! You've successfully authenticated, but GitHub does not provide shell access.` — that message is the expected result, not an error.

If you prefer HTTPS to SSH, use the second clone command instead and authenticate with your GitHub username and a personal access token *that you generate for your own account* — created under *Settings* → *Developer settings* → *Personal access tokens*, scoped to repository read access, and never shared with anyone else or written into a document.

8.3 Building on Ubuntu 20.04

If the lab PC is on Ubuntu 20.04 (the officially supported target for ROS Noetic), building is standard:

```
cd ~/carolus_ws
catkin_make --pkg ff_msgs # build this dependency first, avoids ordering
                           issues
catkin_make
source devel/setup.bash
```

Skip the rest of this section and go to Section 8.5.

8.4 Building on Ubuntu 22.04 (unsupported, requires fixes)

ROS Noetic officially targets Ubuntu 20.04 only. If the lab PC runs 22.04, four system-level fixes are needed before `catkin_make` succeeds, and a fifth fix is needed at launch time.

8.4.1 Fix 1: glibc symlinks

`librt`, `libpthread`, and `libdl` were merged into `libc.so.6` starting with glibc 2.34 (shipped in 22.04); older binaries and build systems still look for the separate files.

```

sudo ln -sf /usr/lib/x86_64-linux-gnu/libc.so.6 /usr/lib/x86_64-linux-gnu/
librt.so
sudo ln -sf /usr/lib/x86_64-linux-gnu/libc.so.6 /usr/lib/x86_64-linux-gnu/
libpthread.so
sudo ln -sf /usr/lib/x86_64-linux-gnu/libc.so.6 /usr/lib/x86_64-linux-gnu/
libdl.so

```

8.4.2 Fix 2: libpython3.8 symlink

Referenced by rospack (an official ROS Noetic package):

```

sudo ln -sf /usr/lib/x86_64-linux-gnu/libpython3.8.so.1 /usr/lib/x86_64-linux-
gnu/libpython3.8.so

```

8.4.3 Fix 3: Boost 1.71 (real libraries, not symlinks)

ROS Noetic's official binaries need Boost 1.71; 22.04 ships 1.74, which is ABI-incompatible for the libraries ROS actually links (regex, filesystem). Symlinking will not work here — the real 1.71 .so files are needed:

```

# Download from http://old-releases.ubuntu.com/ubuntu/pool/{main,universe}/b/
boost1.71/
# the following packages: libboost-{chrono,date-time,filesystem,program-
options,
# regex,system,thread}1.71.0

dpkg-deb -x <package>.deb <extract_dir>
# copy each extracted .so.1.71.0 file into /usr/lib/x86_64-linux-gnu/
sudo ldconfig

```

Pitfall: if a symlink already exists at the destination path (pointing at the 1.74 library), a plain `cp` writes *through* the symlink and corrupts the real system 1.74 file. Always remove the symlink first:

```

sudo rm -f /usr/lib/x86_64-linux-gnu/libboost_<name>.so.1.74.0    # if it is a
symlink
# then copy the real 1.71 file to that path

```

If this happens anyway, recovery is a straightforward package reinstall:

```

sudo apt install --reinstall libboost-{chrono,date-time,filesystem,program-
options,regex,system,thread}1.74.0

```

8.4.4 Fix 4: ICU 67

A dependency of Boost 1.71's regex component; 22.04 ships 66/70, not 67.

```

# Download libicu67 from old-releases.ubuntu.com
# copy libicuuc.so.67*, libicui18n.so.67*, libicudata.so.67* into /usr/lib/
x86_64-linux-gnu/
sudo ldconfig

```

8.4.5 Building, after the four fixes

```
cd ~/carolus_ws
rm -rf build/ devel/
catkin_make --pkg ff_msgs
catkin_make
source devel/setup.bash
```

8.4.6 Fix 5: LD_PRELOAD at launch (a runtime fix, not a build fix)

A binary compiled on 22.04 links against `liblog4cxx.so.12` and `libtinyxml2.so.9`; ROS Noetic's own libraries (`libroscconsole.so`, `libimage_transport.so`) expect `.so.10/.so.6`. Both versions load at runtime and the 22.04 version wins global symbol resolution, which segfaults inside `ros::console::initialize()`. The fix, scoped to the affected node in its launch file (already applied in the launch file below, and in `ekf_lite.launch`):

```
1 <env name="LD_PRELOAD"
2   value="/usr/lib/x86_64-linux-gnu/liblog4cxx.so.10:/usr/lib/x86_64-linux-
   gnu/libtinyxml2.so.6" />
```

Apply the same pattern to any other C++ node recompiled under 22.04 that segfaults the same way on startup.

8.4.7 If an additional ROS package needs to be built from source

Packages unavailable via `apt` on 22.04 (e.g. `robot_localization`, for planned EKF sensor-fusion work) can fail with CMake errors about a missing `<pkg>Config.cmake` or a nonexistent `orocos_kdl` include path — seven packages (`eigen_conversions`, `tf2_geometry_msgs`, `kdl_parser`, `kdl_conversions`, `robot_state_publisher`, `tf2_kdl`, `tf_conversions`) are broken this way by the 20.04→22.04 packaging difference. Without root access to fix the real files under `/opt/ros/noetic/`, the working solution is a set of locally corrected copies in `carolus_ws/cmake_shims/<pkg>_fix/`, given priority via `list(PREPEND CMAKE_PREFIX_PATH ...)` in the `CMakeLists.txt` of whichever package is being built.

Stale-cache pitfall: if a shim is created only after a first failed build attempt, `<pkg>_DIR` may stay cached with the old broken path. Clear just that variable rather than wiping the whole build:

```
cmake -U <pkg>_DIR
```

8.5 Launching Carolus

Carolus's parameters are split in two, so the node can run on other robots without editing any code:

- `src/carolus_node/config/robomaster_s1.yaml` — everything specific to *this* robot and *this* beacon: camera intrinsics, distortion, beacon geometry, blob-detection tuning.
- `src/carolus_node/launch/carolus.launch` — the generic launch file. It contains no hardware values at all; it loads a profile and wires up topics.

`testcarolus.launch` is the RoboMaster S1 entry point: it selects the S1 profile, adds an S1-specific static transform, and delegates to `carolus.launch`.

8.5.1 The hardware profile

The values below are this robot's. **Every one of them must be re-measured for different hardware** — see Chapter 7 for the camera intrinsics, and the note on `known_points` at the end of this chapter for the beacon.

```
1 # --- Camera intrinsics (MEASURE THESE FOR YOUR OWN CAMERA) ---
2 fx: 546.1957
3 fy: 547.0838
4 cx: 575.6041
5 cy: 372.1876
6 distortion: "-0.1479, 0.1452, -0.0023, 0.0025, 0.0070" # k1 k2 p1 p2 k3
7
8 # --- Beacon geometry (MEASURE THIS FOR YOUR OWN BEACON) ---
9 # Four LED positions in metres, Carolus's left-handed frame. One point is
10 # deliberately off the plane of the other three -- P4P needs that.
11 known_points: "0.0825 0.0 0.0 -0.0825 0.0 0.0 0.0 0.072 0.0 0.0 0.0 0.0555"
12
13 # --- Blob detection ---
14 min_circularity: 0.6
15 saturation_threshold: 80
16 min_area: 8.0
17 max_area: 1800.0
18 max_distance_lim: 1000.0
19 lb_hue: 90.0 # OpenCV 0-180 scale, NOT 0-360. Blue LEDs here.
20 ub_hue: 140.0
21
22 # --- Preprocessing ---
23 kernel_size_gaussian: 3
24 kernel_size_morph: 3
25 image_threshold: 190
26
27 # --- Outlier rejection (moving window over successive poses) ---
28 filter_size: 7
29 translation_threshold: 0.5
30 rotation_threshold: 0.3
31 fifo_on: true
32
33 # --- Execution ---
34 num_threads: 1
35 queue_size: 10
36 benchtest: true
37
38 # --- Camera selection (Astrobee-era flags, kept for compatibility) ---
39 nav_cam: true
40 dock_cam: false
41 fisheye: false
42 fov: true
43 mono: false
44 bot_name: "wannabee" # inert legacy label, no functional effect
```

8.5.2 Launching

On this robot, nothing changes — the entry point is the same as before:

```
roslaunch carolus_node testcarolus.launch
```

On Ubuntu 20.04, including the Raspberry Pi, disable the LD_PRELOAD workaround: it exists only to work around the 22.04 library mismatch described in Fix 5 above, and is unnecessary — and wrong-architecture — anywhere else:

```
roslaunch carolus_node testcarolus.launch ubuntu2204_preload:=false
```

On a different robot entirely, copy the profile, substitute your own measured values, and point Carolus at your own camera topic. No source file and no launch file needs editing:

```
roslaunch carolus_node carolus.launch \  
  config:=$(rospack find carolus_node)/config/my_robot.yaml \  
  camera_topic:=/my/camera/image_raw \  
  ubuntu2204_preload:=false
```

The `robomaster_cam` package (Chapter 6) is the DJI SDK bridge for this specific robot and is **not** needed on other hardware — supply the camera stream however your own platform already does.

Launch it (after sourcing the workspace):

```
roslaunch carolus_node testcarolus.launch
```

8.6 Verifying it is running

```
roslaunch carolus_node testcarolus.launch  
rosnode info /carolus_astrobeerex  
rostopic hz /pose  
rostopic hz /postprocessed/image
```

With a beacon visible to the camera, `/pose` publishes a `geometry_msgs/PoseStamped`. Visualize blob detection in RViz by adding an **Image** display on `/postprocessed/image`: correctly detected blobs appear as small circles over the live feed.

8.7 Beacon geometry and blob-filtering parameters

The parameters most likely to need tuning for a new beacon or lighting condition:

Parameter	Meaning
<code>min_circularity</code>	Rejects non-round contours; closer to 1 is stricter.
<code>saturation_threshold</code>	Rejects low-saturation pixels (in HSV) before hue filtering.
<code>min_area, max_area</code>	Rejects contours outside this pixel-area range.
<code>max_distance_lim</code>	Rejects candidate 4-blob groups whose pairwise distances exceed this.
<code>lb_hue, ub_hue</code>	Hue range the beacon’s LEDs must fall within, in OpenCV’s 0–180 scale , not 0–360.

The hue scale is worth stating explicitly, because getting it wrong silently kills all detection. OpenCV stores hue on 0–180 (half the usual 0–360 degree circle, so it fits in one byte), and the node compares the blob’s mean hue against `lb_hue/ub_hue` on that same 0–180 scale. Rough reference points: red $\approx$ 0–10 and 170–180, green $\approx$ 50–80, blue $\approx$ 100–140. This project’s beacon uses blue LEDs, hence the 90–140 window in the launch file. If you enter a value from the 0–360 convention (e.g. 240 for blue), every blob is rejected and the node runs normally while never publishing a pose.

`known_points` is the beacon’s four LED positions in metres, in Carolus’s own left-handed frame, with one point deliberately off the plane of the other three (needed for P4P to resolve pose unambiguously). **These are a physical measurement of this project’s specific beacon, not a universal constant** — the same caveat as the camera intrinsics in Chapter 7. A beacon built to different dimensions needs these four points re-measured and substituted; reusing this project’s values against a differently-sized beacon will make the P4P solve fail silently or produce a plausible-looking but wrong pose.